

Dan, Mike, and Ulrik: Operational and Theoretical Frameworks for Simplicial and Directed Type Theory

Groupoid Infinity

May 2026

Abstract

This article presents the design, syntax, and semantics of three interrelated languages: **Dan**, **Mike**, and **Ulrik**. Together, they form a multi-layered typechecking and compilation system for simplicial and directed homotopy type theories. **Dan** acts as a presentation-oriented algebraic front-end; **Mike** implements a Directed Type Theory (**DTT**) with ends, coends, and path induction; and **Ulrik** implements a Simplicial Type Theory (**STT**) with extension types, twisted arrow categories, and type-level modalities.

Contents

1 Introduction

The verification of higher-dimensional structures and category theory in proof assistants is notoriously difficult due to the "infinite stack of coherences" required by ∞ -categories. Modern synthetic approaches address this by internalizing geometric structures into type theory.

The system described here is comprised of three languages:

1. **Dan**: An operational, presentation-oriented front-end language designed for computer algebra and simplicial complexes. It acts as a bridge between combinatorial structures and type theory.
2. **Mike**: A Directed Type Theory (**DTT**) interpreter implementing first-order category theory, hom-types, path-induction primitives (J , J_{cov} , J_{contra}), and integrals (ends and coends).
3. **Ulrik**: A Simplicial Type Theory (**STT**) proof assistant based on Riehl-Shulman and Gratzer-Weinberger-Buchholtz formulations, implementing a directed interval $\mathbb{I}$, extension types, opposite category modalities (A^{op}), and twisted arrow category modalities (A^{tw}).

The compiler bridge, **lift**, elaborates and translates operational **Dan** files into theoretical **Ulrik** (STT mode) or **Mike** (DTT mode) files.

1.1 Installation and Usage

The compilers are implemented in OCaml and can be built using the Dune build system:

```
$ dune build
```

To typecheck a library file in Simplicial Type Theory mode:

```
$ dune exec ulrik library/ulrik/book.ulrik
```

To typecheck a library file in Directed Type Theory mode:

```
$ dune exec mike library/mike/book.mike
```

The lifting tool can translate operational code:

```
$ dune exec lift -stt library/dan/simplex.dan
```

```
$ dune exec lift -dirtt library/dan/simplex.dan
```

2 Syntax

2.1 Abstract Syntax Tree (AST)

The core AST of the theoretical typechecker is defined uniformly in `syntax.ml`. The AST includes common MLTT term formers, Simplicial STT primitives, and compatibility DTT constructors:

```
type expr =
  (* Common MLTT *)
  | EVar of name
  | EApp of expr * expr
  | ELam of (name * expr option) * expr
  | EPi of expr * (name * expr)
  | ESig of expr * (name * expr)
  | EPair of expr * expr
  | EFst of expr
  | ESnd of expr
  | EUniv
  | EId of expr * expr * expr
  | ERef of expr

  (* U *)
  (* strict Id *)
  (* refl *)

  (* Simplicial Core (STT) *)
  | EIDir
  | EZeroDir
  | EOneDir
  | ELeq of expr * expr
  | EShapeInc of expr * expr
  | EExt of expr * expr * expr
  | ESystem of (expr * expr) list
  | EModalPi of expr * (name * expr)
  | EModalLam of (name * expr) * expr
  | EModalApp of expr * expr
  | ETw of expr
  | EOp of expr
  | ETwPi0 of expr
  | ETwPi1 of expr
  | EJoin of expr * expr
  | EMeet of expr * expr
  | ENeg of expr

  (* directed interval I *)
  (* 0 *)
  (* 1 *)
  (* i <= j *)
  (* phi <= psi *)
  (* {x : A / ^ phi f} *)
  (* [ phi | f ] ... *)
  (* mu(x:A). B(x) *)
  (* lambda^mu(x:A). body *)
  (* f @ phi *)
  (* A^tw *)
  (* category opposite *)
  (* pi0(x) *)
  (* pi1(x) *)
  (* i \ / j *)
  (* i / \ j *)
  (* not i *)
```

```

(* Dirtt Compatibility (DTT) *)
| EHom of expr * expr * expr (* hom(cat, a, b) *)
| ETensor of expr * expr (* M * N *)
| EFunc of expr * expr (* M → N *)
| ECoend of expr * name * expr (* coend(x:cat). M *)
| EEnd of expr * name * expr (* end(x:cat). M *)
| EIdTerm of expr (* id(a) *)
| EJ of expr * name * name * name * expr * expr * expr * expr
| EJCov of expr * name * expr * expr * expr
| EJContra of expr * name * expr * expr * expr
| ETensorElim of name * name * expr * expr
| ECoendIntro of name * name * name * expr
| ECoendElim of name * name * expr * expr
| EEndElim of name * name * name * expr * expr

```

2.2 Top-Level Commands

The surface syntax processes files as lists of commands:

```

type cmd =
| CModule of string
| CImport of string
| CFunctor of name * (expr * sign) list * expr
| CDef of name * name list * expr option * expr
| CCheck of (name * expr) list * (name * expr) list * expr * expr
| CCheckSimplicial of (name * expr) list * expr * expr

```

3 Semantics

3.1 Presentation-Oriented Semantics of Dan

The operational language **Dan** acts as a presentation-oriented algebraic and simplicial front-end. It differs fundamentally from **Mike** and **Ulrik** in that it is designed for static presentation verification rather than general proof search or type checking.

3.1.1 Unified Sequent Presentations

A construction in **Dan** is presented using a unified algebraic sequent:

$$\Pi(\text{context}), \text{conditions} \vdash \text{rank}(\text{generators} \mid \text{relations}) \quad (1)$$

The context declares elements of the universe of simplicial shapes (e.g. **Simplex**), and the relations restrict their boundary faces. The rank verifies that the list of generators matches the dimension of the presentation.

3.1.2 Algebraic Domains

Dan provides built-in presentation checkers for several categories of algebraic and higher-dimensional structures:

- **Groups and Monoids:** Defined by generators and multiplication/identity relations. For example, the cyclic group $\mathbb{Z}/3\mathbb{Z}$ is presented by:

$$\text{def } z3 : \text{Group} := \Pi (e \ a : \text{Simplex}), \ a^3 = e \vdash 1 \ (a \mid a^3 = e)$$

- **Rings and Fields:** Defined by Prime Field or Polynomial relations, checked in linear time using internal lookup tables.
- **Simplicial Sets:** Defined by vertices, edges, face operators (∂_{ij}), and degeneracy relations (s_i). For example, the simplicial circle S^1 is presented as:

$$\text{def } \text{circle} : \text{Simplicial} := \Pi (v \ e : \text{Simplex}), \ \partial_{10} = v, \ \partial_{11} = v, \ s_0 <$$

- **Categories:** Presented by objects, morphisms, and composition constraints ($f \circ g = h$).

3.1.3 Elaboration and Mapping

When a presentation in **Dan** is processed by the `lift` compiler, it is mapped into the theoretical types of either **Ulrik** (**STT**) or **Mike** (**DTT**):

- Generators and simplex parameters map to points/types.
- Edges and category morphisms map to paths over the directed interval:

$$\text{hom}(C, x, y) \equiv \{t : \mathbb{I} \rightarrow C \mid t(0) = x, t(1) = y\} \quad (2)$$

- Presentation relations map to Segal equivalence paths or path-induction properties.

3.2 Directed Type Theory (Mike)

Directed Type Theory (**DTT**) treats categories as first-order structures and incorporates multivariable category theory (Schultz-Shulman Kits).

3.2.1 Path Induction

Morphisms in **DTT** are treated as directed paths. Path induction is realized via directed J operators:

- J_{cov} and J_{contra} perform path induction along covariant and contravariant positions.
- Composition of $f : \text{hom}(A, x, y)$ and $g : \text{hom}(A, y, z)$ is defined via path induction:

$$g \circ f \equiv J(w_1.w_2.\text{hom}(A, x, w_2), z, f, y, z, g) \quad (3)$$

3.3 Simplicial Type Theory (Ulrik)

Simplicial Type Theory (**STT**) internalizes simplicial sets by extending dependent type theory with a directed interval and extension types.

3.3.1 The Directed Interval $\mathbb{I}$

The directed interval $\mathbb{I}$ is a pretype representing the simplicial 1-simplex Δ^1 . It behaves as a De Morgan algebra:

- Global endpoints $0, 1 : \mathbb{I}$ representing the source and target.
- A preorder relation $\leq$ such that $0 \leq t \leq 1$.
- Lattice operations: meet ($\wedge$), join ($\vee$), and negation ($\neg$).

3.3.2 Cofibrations and Shapes

Shapes (or cofibrations) are propositions that classify sub-complexes of simplicial cubes. For instance, the boundary of the interval $\partial\mathbb{I}$ is the shape $t \leq 0 \vee 1 \leq t$.

3.3.3 Extension Types

Given a type A , a shape φ , and a boundary function $f : \varphi \rightarrow A$, the extension type $\{x : A \mid \varphi f\}$ represents the type of elements of A that restrict definitionally to f when φ holds. The morphism type $\text{hom}(C, x, y)$ is defined as the extension type of paths $\mathbb{I} \rightarrow C$ restricting to x at 0 and y at 1:

$$\text{hom}(C, x, y) \equiv \{p : \mathbb{I} \rightarrow C \mid t \leq 0 \vee 1 \leq t [t \leq 0 \mapsto x, 1 \leq t \mapsto y]\} \quad (4)$$

3.3.4 Twisted Arrow Modality A^{tw}

The twisted arrow category is constructed using the modality A^{tw} . Elements of A^{tw} represent morphisms in A , projected via $\pi_0 : A^{\text{tw}} \rightarrow A^{\text{op}}$ and $\pi_1 : A^{\text{tw}} \rightarrow A$.

3.3.5 Integrals: Ends and Coends

Limits and colimits are represented via ends $(\int_{x:A} T(x))$ and coends $(\int^{x:A} T(x))$ constructions.

- Natural transformations are ends over the homs:

$$\text{Nat}(F, G) \equiv \int_{w:A} \text{hom}(B, F(w), G(w)).$$

- Colimits are coends:

$$\text{Colimit}(F) \equiv \int^{w:A} \text{hom}(A, w, w)$$

4 Properties

4.1 Definitional Duality

In both **Ulrik** and **Mike**, the opposite category operation $(\cdot)^{\text{op}}$ is a definitional involution:

$$(C^{\text{op}})^{\text{op}} \equiv C \quad (5)$$

This allows dual theorems to be proven "for free" without requiring explicit transport along isomorphisms.

4.2 Type Inference

The unified definition command `def name := body` derives the type of ‘body’ using type inference in **STT** mode. For a type-level term (where ‘body’ represents a type), the checker infers ‘U’ as its type and stores it as a type definition in the global context.

4.3 Naturality and Yoneda Lemma

In **STT**, all functions are automatically functorial. A natural transformation between presheaves $F, G : C \rightarrow \text{Space}$ is equivalent to a dependent function:

$$\text{PshNat}(F, G) \equiv (w : C) \rightarrow F(w) \rightarrow G(w) \quad (6)$$

The Yoneda Lemma maps natural transformations from the representable functor $\text{Yo}(c) \equiv \lambda c'. \text{hom}(C, c, c')$ to an arbitrary presheaf F :

$$\text{yoneda_map} : \text{PshNat}(\text{Yo}(c), F) \rightarrow F(c) \quad (7)$$

The map evaluates the natural transformation η at the identity path:

$$\text{yoneda_map}(\eta) \equiv \eta(c, \lambda t. c) \quad (8)$$

5 Annex A. Simplicial Type Theory Equations

5.1 Evaluation of Extension Types

$$\text{eval}(\text{EExt}(a, \varphi, f)) \equiv \text{VExt}(\text{eval}(a), \text{VShapeClosure}(\text{env}, \varphi), \text{VShapeClosure}(\text{env}, f)) \quad (9)$$

5.2 Checking of Extension Types

To check a term e against $\text{VExt}(\text{base_tp}, \varphi, f)$: 1. Verify $\text{check}(\text{ctx}, \text{env}, e, \text{base_tp})$ where base_tp is $\Pi(t : \mathbb{I}).B(t)$. 2. Evaluate the shape boundary φ and function f in their respective closure environments:

$$\varphi_{val} \equiv \text{eval}(\text{ctx}, \text{val_env} \cup \text{env}_\varphi, \varphi_{exp}) \quad (10)$$

$$f_{val} \equiv \text{eval}(\text{ctx}, \text{val_env} \cup \text{env}_f, f_{exp}) \quad (11)$$

3. Assert that for all valuations where $\varphi_{val} \equiv 1$, the application $e(t)$ is equivalent to f_{val} .